短信转发助手 - 源程序鉴别材料

软件基本信息

- **软件名称**：短信转发助手
- **版本号**：V2.7.2
- **著作权人**：华昊科技有限公司
- **开发者**：王士辉
- **联系邮箱**：huahao@email.cn
- **开发完成日期**：2026年4月5日

源程序量

约 **4000 行**（仅 Android 主程序 Kotlin 代码）

第一部分：前 30 页

文件 1：MainActivity.kt（前 600 行）

```kotlin
/*
 * 短信转发助手
 * 版本：V2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.Manifest
import android.app.Activity
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.graphics.Color as AndroidColor
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.os.PowerManager
import android.provider.Settings
```

```
import android.telephony.SubscriptionManager
import android.telephony.TelephonyManager
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.*
import androidx.compose.animation.core.*
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.content.ContextCompat
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import org.json.JSONArray
import org.json.JSONObject
import java.io.File

class MainActivity : ComponentActivity() {

    private lateinit var requestPermissionLauncher: ActivityResultLauncher<Array<String>>
    private lateinit var requestBatteryOptimizationLauncher: ActivityResultLauncher<Intent>

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        requestPermissionLauncher = registerForActivityResult(
            ActivityResultContracts.RequestMultiplePermissions()
```

```
        ) { permissions ->
            val allGranted = permissions.values.all { it }
            if (allGranted) {
                checkBatteryOptimizationAndStartService()
            } else {
                Toast.makeText(this, "请授予所有权限", Toast.LENGTH_LONG).show()
            }
        }

        requestBatteryOptimizationLauncher = registerForActivityResult(
            ActivityResultContracts.StartActivityForResult()
        ) {
            startServiceIfEnabled()
        }

        checkPermissionsAndStartService()

        setContent {
            App()
        }
    }

    private fun checkPermissionsAndStartService() {
        val requiredPermissions = mutableListOf(
            Manifest.permission.RECEIVE_SMS,
            Manifest.permission.READ_PHONE_STATE
        )
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            requiredPermissions.add(Manifest.permission.POST_NOTIFICATIONS)
        }

        val missingPermissions = requiredPermissions.filter {
            ContextCompat.checkSelfPermission(this, it) != PackageManager.PERMISSION_GRANTED
        }

        if (missingPermissions.isNotEmpty()) {
            requestPermissionLauncher.launch(missingPermissions.toTypedArray())
        } else {
            checkBatteryOptimizationAndStartService()
        }
    }

    private fun checkBatteryOptimizationAndStartService() {
        val pm = getSystemService(Context.POWER_SERVICE) as PowerManager
        if (!pm.isIgnoringBatteryOptimizations(packageName)) {
            val intent = Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS).apply
{
                data = Uri.parse("package:$packageName")
            }
            requestBatteryOptimizationLauncher.launch(intent)
        } else {
            startServiceIfEnabled()
        }
```

```
    }

    private fun startServiceIfEnabled() {
        val prefs = getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)
        val enabled = prefs.getBoolean(Constants.PREF_ENABLED, false)
        val startOnBoot = prefs.getBoolean(Constants.PREF_START_ON_BOOT, true)

        if (enabled) {
            val serviceIntent = Intent(this, SmsForegroundService::class.java)
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                startForegroundService(serviceIntent)
            } else {
                startService(serviceIntent)
            }
        }
    }
}

@Composable
fun App() {
    var selectedTab by rememberSaveable { mutableIntStateOf(0) }
    val context = LocalContext.current
    val prefs = remember { context.getSharedPreferences(Constants.PREFS_NAME,
Context.MODE_PRIVATE) }
    var refreshKey by remember { mutableIntStateOf(0) }

    val tabs = listOf(
        "首页" to Icons.Default.Home,
        "关键词" to Icons.Default.Search,
        "通道" to Icons.Default.Send,
        "设置" to Icons.Default.Settings,
        "日志" to Icons.Default.List
    )

    MaterialTheme(
        colorScheme = if (isSystemInDarkTheme()) darkColorScheme() else lightColorScheme()
    ) {
        Scaffold(
            bottomBar = {
                NavigationBar {
                    tabs.forEachIndexed { index, (title, icon) ->
                        NavigationBarItem(
                            selected = selectedTab == index,
                            onClick = { selectedTab = index },
                            icon = { Icon(icon, contentDescription = title) },
                            label = { Text(title) }
                        )
                    }
                }
            },
            topBar = {
                TopAppBar(
                    title = { Text("短信转发助手") },
```

```
                    actions = {
                        var showAbout by remember { mutableStateOf(false) }

                        IconButton(onClick = { showAbout = true }) {
                            Icon(Icons.Default.Info, contentDescription = "关于")
                        }

                        if (showAbout) {
                            AboutDialog(
                                onDismiss = { showAbout = false }
                            )
                        }
                    }
                )
            }
        ) { paddingValues ->
            Box(modifier = Modifier.padding(paddingValues)) {
                when (selectedTab) {
                    0 -> HomeTab(
                        prefs = prefs,
                        onRefresh = { refreshKey++ }
                    )
                    1 -> KeywordTab(prefs = prefs)
                    2 -> ChannelTab(prefs = prefs)
                    3 -> SettingsTab(prefs = prefs, context = context)
                    4 -> LogTab(context = context, refreshKey = refreshKey)
                }
            }
        }
    }
}

@Composable
fun HomeTab(prefs: android.content.SharedPreferences, onRefresh: () -> Unit) {
    var enabled by remember { mutableStateOf(prefs.getBoolean(Constants.PREF_ENABLED,
false)) }
    var startOnBoot by remember {
mutableStateOf(prefs.getBoolean(Constants.PREF_START_ON_BOOT, true)) }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        item {
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor = if (enabled) MaterialTheme.colorScheme.primaryContainer
                    else MaterialTheme.colorScheme.errorContainer
                )
            ) {
```

```
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(20.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Icon(
                        imageVector = if (enabled) Icons.Default.CheckCircle else
Icons.Default.Error,
                        contentDescription = null,
                        modifier = Modifier.size(64.dp),
                        tint = if (enabled) MaterialTheme.colorScheme.primary
                        else MaterialTheme.colorScheme.error
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    Text(
                        text = if (enabled) "服务运行中" else "服务已停止",
                        style = MaterialTheme.typography.headlineSmall,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                    Text(
                        text = if (enabled) "短信转发功能已启用" else "点击下方开关启用服务",
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
        }

        item {
            Card(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Default.PowerSettingsNew, contentDescription = null)
                            Spacer(modifier = Modifier.width(12.dp))
                            Text("启用服务", style = MaterialTheme.typography.titleMedium)
                        }
                        Switch(
                            checked = enabled,
                            onCheckedChange = { newEnabled ->
                                enabled = newEnabled
```

```
                                prefs.edit().putBoolean(Constants.PREF_ENABLED,
newEnabled).apply()
                                onRefresh()
                            }
                        )
                    }
                }
            }
        }

        item {
            Card(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Icon(Icons.Default.PhoneAndroid, contentDescription = null)
                            Spacer(modifier = Modifier.width(12.dp))
                            Text("开机自启动", style = MaterialTheme.typography.titleMedium)
                        }
                        Switch(
                            checked = startOnBoot,
                            onCheckedChange = {
                                startOnBoot = it
                                prefs.edit().putBoolean(Constants.PREF_START_ON_BOOT,
it).apply()
                            }
                        )
                    }
                }
            }
        }

        item {
            Card(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Text(
                        "快速操作",
```

```
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(12.dp))
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    Button(
                        onClick = onRefresh,
                        modifier = Modifier.weight(1f)
                    ) {
                        Icon(Icons.Default.Refresh, contentDescription = null)
                        Spacer(modifier = Modifier.width(8.dp))
                        Text("刷新日志")
                    }
                    OutlinedButton(
                        onClick = { /* TODO: 测试规则 */ },
                        modifier = Modifier.weight(1f)
                    ) {
                        Icon(Icons.Default.PlayArrow, contentDescription = null)
                        Spacer(modifier = Modifier.width(8.dp))
                        Text("测试规则")
                    }
                }
            }
        }
    }

    item {
        Card(
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp)
            ) {
                Text(
                    "提示",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(8.dp))
                Text(
                    "• 在「通道」页面添加转发通道\n" +
                    "• 在「关键词」页面配置过滤规则\n" +
                    "• 在「设置」页面配置 SIM 卡信息\n" +
                    "• 在「日志」页面查看转发记录",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
```

```
            }
        }
    }
}

@Composable
fun KeywordTab(prefs: android.content.SharedPreferences) {
    var keywordConfigs by remember {
        mutableStateOf(loadKeywordConfigs(prefs))
    }
    var showAddDialog by remember { mutableStateOf(false) }
    var editingConfig by remember { mutableStateOf<KeywordConfig?>(null) }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        item {
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor = MaterialTheme.colorScheme.primaryContainer
                )
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Icon(Icons.Default.Info, contentDescription = null)
                        Spacer(modifier = Modifier.width(8.dp))
                        Text(
                            "关键词为空时转发所有短信",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onPrimaryContainer
                        )
                    }
                }
            }
        }

        items(keywordConfigs) { config ->
            KeywordConfigCard(
                config = config,
                onEdit = {
                    editingConfig = config
                    showAddDialog = true
                },
                onDelete = {
                    keywordConfigs = keywordConfigs - config
```

```
                    saveKeywordConfigs(prefs, keywordConfigs)
                }
            )
        }

        item {
            Button(
                onClick = {
                    editingConfig = null
                    showAddDialog = true
                },
                modifier = Modifier.fillMaxWidth()
            ) {
                Icon(Icons.Default.Add, contentDescription = null)
                Spacer(modifier = Modifier.width(8.dp))
                Text("添加关键词")
            }
        }
    }

    if (showAddDialog) {
        KeywordEditDialog(
            config = editingConfig,
            onDismiss = { showAddDialog = false },
            onSave = { newConfig ->
                keywordConfigs = if (editingConfig != null) {
                    keywordConfigs.map { if (it.id == editingConfig!!.id) newConfig else it
}
                } else {
                    keywordConfigs + newConfig
                }
                saveKeywordConfigs(prefs, keywordConfigs)
                showAddDialog = false
            }
        )
    }
}

@Composable
fun KeywordConfigCard(
    config: KeywordConfig,
    onEdit: () -> Unit,
    onDelete: () -> Unit
) {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            Row(
```

```
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column(modifier = Modifier.weight(1f)) {
                    Text(
                        text = if (config.keyword.isEmpty()) "全部短信" else config.keyword,
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        text = "通道 ID: ${config.channelId}",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
                Row {
                    IconButton(onClick = onEdit) {
                        Icon(Icons.Default.Edit, contentDescription = "编辑")
                    }
                    IconButton(onClick = onDelete) {
                        Icon(Icons.Default.Delete, contentDescription = "删除", tint =
MaterialTheme.colorScheme.error)
                    }
                }
            }
        }
    }
}

@Composable
fun KeywordEditDialog(
    config: KeywordConfig?,
    onDismiss: () -> Unit,
    onSave: (KeywordConfig) -> Unit
) {
    var keyword by remember { mutableStateOf(config?.keyword ?: "") }
    var channelId by remember { mutableStateOf(config?.channelId ?: "") }
    val context = LocalContext.current
    val prefs = remember { context.getSharedPreferences(Constants.PREFS_NAME,
Context.MODE_PRIVATE) }
    val channels = remember { loadChannels(prefs) }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(if (config == null) "添加关键词" else "编辑关键词") },
        text = {
            Column(
                verticalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                OutlinedTextField(
                    value = keyword,
```

```
                    onValueChange = { keyword = it },
                    label = { Text("关键词（留空转发全部）") },
                    modifier = Modifier.fillMaxWidth(),
                    singleLine = true
                )
                if (channels.isNotEmpty()) {
                    var expanded by remember { mutableStateOf(false) }
                    ExposedDropdownMenuBox(
                        expanded = expanded,
                        onExpandedChange = { expanded = !expanded }
                    ) {
                        OutlinedTextField(
                            value = channels.find { it.id == channelId }?.name ?: channelId,
                            onValueChange = {},
                            readOnly = true,
                            label = { Text("选择通道") },
                            trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
                            modifier = Modifier
                                .fillMaxWidth()
                                .menuAnchor()
                        )
                        ExposedDropdownMenu(
                            expanded = expanded,
                            onDismissRequest = { expanded = false }
                        ) {
                            channels.forEach { channel ->
                                DropdownMenuItem(
                                    text = { Text(channel.name) },
                                    onClick = {
                                        channelId = channel.id
                                        expanded = false
                                    }
                                )
                            }
                        }
                    }
                } else {
                    Text(
                        "请先在「通道」页面添加通道",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.error
                    )
                }
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (channelId.isNotEmpty()) {
                        onSave(
                            KeywordConfig(
                                id = config?.id ?: java.util.UUID.randomUUID().toString(),
```

```
                                keyword = keyword,
                                channelId = channelId
                            )
                        )
                    }
                },
                enabled = channelId.isNotEmpty()
            ) {
                Text("保存")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("取消")
            }
        }
    )
}

@Composable
fun ChannelTab(prefs: android.content.SharedPreferences) {
    var channels by remember { mutableStateOf(loadChannels(prefs)) }
    var showAddDialog by remember { mutableStateOf(false) }
    var editingChannel by remember { mutableStateOf<Channel?>(null) }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        if (channels.isEmpty()) {
            item {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.secondaryContainer
                    )
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(24.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Icon(
                            Icons.Default.Add,
                            contentDescription = null,
                            modifier = Modifier.size(48.dp),
                            tint = MaterialTheme.colorScheme.onSecondaryContainer
                        )
                        Spacer(modifier = Modifier.height(12.dp))
                        Text(
```

```
                        "还没有通道",
                        style = MaterialTheme.typography.titleLarge,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        "点击下方按钮添加转发通道",
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSecondaryContainer
                    )
                }
            }
        }
    }

    items(channels) { channel ->
        ChannelCard(
            channel = channel,
            onEdit = {
                editingChannel = channel
                showAddDialog = true
            },
            onDelete = {
                channels = channels - channel
                saveChannels(prefs, channels)
            }
        )
    }

    item {
        Button(
            onClick = {
                editingChannel = null
                showAddDialog = true
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Icon(Icons.Default.Add, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text("添加通道")
        }
    }
}

if (showAddDialog) {
    ChannelEditDialog(
        channel = editingChannel,
        onDismiss = { showAddDialog = false },
        onSave = { newChannel ->
            channels = if (editingChannel != null) {
                channels.map { if (it.id == editingChannel!!.id) newChannel else it }
            } else {
                channels + newChannel
```

```
                }
                saveChannels(prefs, channels)
                showAddDialog = false
            }
        )
    }
}
```

文件 2：SmsReceiver.kt（全部 586 行）

```
/*
 * 短信转发助手
 * 版本：v2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.Manifest
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.provider.Telephony
import android.telephony.SubscriptionManager
import android.telephony.TelephonyManager
import android.telephony.SmsMessage
import android.util.Log
import androidx.core.content.ContextCompat
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.RequestBody.Companion.toRequestBody
import org.json.JSONObject
import java.util.concurrent.TimeUnit

class SmsReceiver : BroadcastReceiver() {

    companion object {
        private const val TAG = "SmsReceiver"
        private val httpClient = OkHttpClient.Builder()
            .connectTimeout(10, TimeUnit.SECONDS)
```

```
            .readTimeout(20, TimeUnit.SECONDS)
            .writeTimeout(20, TimeUnit.SECONDS)
            .callTimeout(45, TimeUnit.SECONDS)
            .build()
    }

    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != Telephony.Sms.Intents.SMS_RECEIVED_ACTION) return

        val prefs = context.getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)
        if (!prefs.getBoolean(Constants.PREF_ENABLED, false)) return

        if (ContextCompat.checkSelfPermission(
                context,
                Manifest.permission.RECEIVE_SMS
            ) != PackageManager.PERMISSION_GRANTED
        ) return

        val messages = Telephony.Sms.Intents.getMessagesFromIntent(intent)
        if (messages == null || messages.isEmpty()) return

        var sender = ""
        val sb = StringBuilder()
        var subscriptionId = -1

        for (sms in messages) {
            sender = sms.displayOriginatingAddress ?: sender
            sb.append(sms.displayMessageBody)
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP_MR1) {
                subscriptionId = sms.subscriptionId
            }
        }

        val content = sb.toString()
        if (content.isEmpty()) return

        val scope = CoroutineScope(Dispatchers.IO)
        scope.launch {
            try {
                processSms(context, prefs, sender, content, subscriptionId)
            } catch (e: Exception) {
                Log.e(TAG, "Error processing SMS", e)
            }
        }
    }

    private suspend fun processSms(
        context: Context,
        prefs: android.content.SharedPreferences,
        sender: String,
        content: String,
        subscriptionId: Int
    ) {
```

```
        val channels = loadChannels(prefs)
        val keywordConfigs = loadKeywordConfigs(prefs)

        if (channels.isEmpty()) return

        val simSlot = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP_MR1) {
            getSimSlotFromSubscriptionId(context, subscriptionId)
        } else {
            1
        }

        val receiverPhone = getReceiverPhoneNumber(context, prefs, subscriptionId, simSlot)
        val showReceiverPhone = prefs.getBoolean(Constants.PREF_SHOW_RECEIVER_PHONE, true)
        val showSenderPhone = prefs.getBoolean(Constants.PREF_SHOW_SENDER_PHONE, true)
        val highlightCode = prefs.getBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE,
true)

        val message = buildMessage(
            content = content,
            sender = sender,
            receiverPhone = receiverPhone,
            showReceiverPhone = showReceiverPhone,
            showSenderPhone = showSenderPhone,
            highlightCode = highlightCode
        )

        val channelsToUse = getMatchingChannels(channels, keywordConfigs, content)

        if (channelsToUse.isEmpty()) {
            Log.d(TAG, "No matching channels, not forwarding")
            return
        }

        for (channel in channelsToUse) {
            sendToChannel(channel, message, sender, content)
        }

        LogStore.appendLog(
            context,
            "Forwarded SMS from $sender to ${channelsToUse.size} channels"
        )
    }

    private fun getSimSlotFromSubscriptionId(context: Context, subscriptionId: Int): Int {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.LOLLIPOP_MR1) return 1
        if (subscriptionId == -1) return 1

        return try {
            val subscriptionManager = ContextCompat.getSystemService(
                context,
                SubscriptionManager::class.java
            ) ?: return 1
```

```
            val activeSubscriptions = subscriptionManager.activeSubscriptionInfoList
            for (info in activeSubscriptions) {
                if (info.subscriptionId == subscriptionId) {
                    return info.simSlotIndex + 1
                }
            }
            1
        } catch (e: Exception) {
            Log.e(TAG, "Error getting sim slot", e)
            1
        }
    }

    private fun getReceiverPhoneNumber(
        context: Context,
        prefs: android.content.SharedPreferences,
        subscriptionId: Int,
        simSlot: Int
    ): String? {
        val customSim1Phone = prefs.getString(Constants.PREF_CUSTOM_SIM1_PHONE, "")?.trim()
        val customSim2Phone = prefs.getString(Constants.PREF_CUSTOM_SIM2_PHONE, "")?.trim()

        if (simSlot == 2 && !customSim2Phone.isNullOrEmpty()) {
            return customSim2Phone
        }
        if (simSlot == 1 && !customSim1Phone.isNullOrEmpty()) {
            return customSim1Phone
        }

        if (!customSim1Phone.isNullOrEmpty()) {
            return customSim1Phone
        }
        if (!customSim2Phone.isNullOrEmpty()) {
            return customSim2Phone
        }

        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.LOLLIPOP_MR1) return null
        if (ContextCompat.checkSelfPermission(
                context,
                Manifest.permission.READ_PHONE_STATE
            ) != PackageManager.PERMISSION_GRANTED
        ) return null

        return try {
            val telephonyManager = ContextCompat.getSystemService(
                context,
                TelephonyManager::class.java
            ) ?: return null

            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP_MR1 && subscriptionId
!= -1) {
                val subscriptionManager = ContextCompat.getSystemService(
                    context,
```

```
                SubscriptionManager::class.java
            )
            if (subscriptionManager != null) {
                val phone = subscriptionManager.getPhoneNumber(subscriptionId)
                if (!phone.isNullOrEmpty()) {
                    return phone
                }
            }
        }

        telephonyManager.line1Number
    } catch (e: Exception) {
        Log.e(TAG, "Error getting phone number", e)
        null
    }
}

private fun buildMessage(
    content: String,
    sender: String,
    receiverPhone: String?,
    showReceiverPhone: Boolean,
    showSenderPhone: Boolean,
    highlightCode: Boolean
): String {
    val code = if (highlightCode) extractVerificationCode(content) else null
    val sb = StringBuilder()

    if (code != null) {
        sb.append("验证码: ").append(code).append("\n")
    }
    if (showReceiverPhone && !receiverPhone.isNullOrEmpty()) {
        sb.append("本机: ").append(receiverPhone).append("\n")
    }
    if (showSenderPhone) {
        sb.append("来自: ").append(sender).append("\n")
    }
    sb.append("\n").append(content)

    return sb.toString()
}

private fun extractVerificationCode(content: String): String? {
    val patterns = listOf(
        "(?:验证码|校验码|验证码是|校验码是|verification|code)[:: \\s]*([0-9]{4,8})",
        "([0-9]{4,8})[是为]*验证码",
        "([0-9]{4,8})[是为]*校验码"
    )

    for (pattern in patterns) {
        val regex = Regex(pattern, RegexOption.IGNORE_CASE)
        val match = regex.find(content)
        if (match != null && match.groupValues.size > 1) {
```

```
                return match.groupValues[1]
            }
        }
        return null
    }

    private fun getMatchingChannels(
        channels: List<Channel>,
        keywordConfigs: List<KeywordConfig>,
        content: String
    ): List<Channel> {
        if (keywordConfigs.isEmpty()) {
            return channels
        }

        val matchingConfigs = keywordConfigs.filter { config ->
            config.keyword.isEmpty() || content.contains(config.keyword, ignoreCase = true)
        }

        if (matchingConfigs.isEmpty()) {
            return emptyList()
        }

        val channelIds = matchingConfigs.map { it.channelId }.toSet()
        return channels.filter { channelIds.contains(it.id) }
    }

    private suspend fun sendToChannel(
        channel: Channel,
        message: String,
        sender: String,
        content: String
    ) {
        try {
            val (requestBody, url) = when (channel.type) {
                ChannelType.WECHAT -> {
                    val json = JSONObject().apply {
                        put("msgtype", "text")
                        put("text", JSONObject().put("content", message))
                    }
                    json.toString().toRequestBody("application/json".toMediaType()) to
channel.target
                }
                ChannelType.DINGTALK -> {
                    val json = JSONObject().apply {
                        put("msgtype", "text")
                        put("text", JSONObject().put("content", message))
                    }
                    json.toString().toRequestBody("application/json".toMediaType()) to
channel.target
                }
                ChannelType.FEISHU -> {
                    val json = JSONObject().apply {
```

```
                        put("msg_type", "text")
                        put("content", JSONObject().put("text", message))
                    }
                    json.toString().toRequestBody("application/json".toMediaType()) to
channel.target
                }
                ChannelType.GENERIC_WEBHOOK -> {
                    message.toRequestBody("text/plain; charset=utf-8".toMediaType()) to
channel.target
                }
            }

            val request = Request.Builder()
                .url(url)
                .post(requestBody)
                .build()

            val response = httpClient.newCall(request).execute()
            if (!response.isSuccessful) {
                Log.e(TAG, "Failed to send to ${channel.name}: ${response.code}")
            } else {
                Log.d(TAG, "Sent to ${channel.name} successfully")
            }
            response.close()
        } catch (e: Exception) {
            Log.e(TAG, "Error sending to ${channel.name}", e)
        }
    }
}

fun loadChannels(prefs: android.content.SharedPreferences): List<Channel> {
    val json = prefs.getString(Constants.PREF_CHANNELS, "[]") ?: "[]"
    val array = JSONArray(json)
    val channels = mutableListOf<Channel>()
    for (i in 0 until array.length()) {
        val obj = array.getJSONObject(i)
        channels.add(
            Channel(
                id = obj.getString("id"),
                name = obj.getString("name"),
                type = ChannelType.valueOf(obj.getString("type")),
                target = obj.getString("target")
            )
        )
    }
    return channels
}

fun saveChannels(prefs: android.content.SharedPreferences, channels: List<Channel>) {
    val array = JSONArray()
    for (channel in channels) {
        val obj = JSONObject().apply {
            put("id", channel.id)
```

```
            put("name", channel.name)
            put("type", channel.type.name)
            put("target", channel.target)
        }
        array.put(obj)
    }
    prefs.edit().putString(Constants.PREF_CHANNELS, array.toString()).apply()
}

fun loadKeywordConfigs(prefs: android.content.SharedPreferences): List<KeywordConfig> {
    val json = prefs.getString(Constants.PREF_KEYWORD_CONFIGS, "[]") ?: "[]"
    val array = JSONArray(json)
    val configs = mutableListOf<KeywordConfig>()
    for (i in 0 until array.length()) {
        val obj = array.getJSONObject(i)
        configs.add(
            KeywordConfig(
                id = obj.getString("id"),
                keyword = obj.getString("keyword"),
                channelId = obj.getString("channelId")
            )
        )
    }
    return configs
}

fun saveKeywordConfigs(prefs: android.content.SharedPreferences, configs:
List<KeywordConfig>) {
    val array = JSONArray()
    for (config in configs) {
        val obj = JSONObject().apply {
            put("id", config.id)
            put("keyword", config.keyword)
            put("channelId", config.channelId)
        }
        array.put(obj)
    }
    prefs.edit().putString(Constants.PREF_KEYWORD_CONFIGS, array.toString()).apply()
}
```

文件 3：Constants.kt（全部 58 行）

```
/*
 * 短信转发助手
 * 版本：V2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
```

```
 */

package com.lanbing.smsforwarder

/**
 * Application constants.
 */
object Constants {
    // Logging
    const val LOG_FILE_NAME = "sms_forwarder_logs.txt"
    const val MAX_LOG_ENTRIES = 200
    const val MAX_LOG_LINE_LENGTH = 2000

    // Deduplication
    const val DUPLICATE_WINDOW_MS = 5000L

    // Retry
    const val MAX_RETRY_ATTEMPTS = 3
    const val INITIAL_RETRY_BACKOFF_MS = 2000L
    const val MAX_FAILED_MESSAGES = 100
    const val FAILED_MESSAGES_FILE = "failed_messages.json"

    // Threading
    const val THREAD_POOL_SIZE = 4
    const val BROADCAST_TIMEOUT_SECONDS = 45L

    // Notification
    const val NOTIFICATION_UPDATE_THROTTLE_MS = 1000L
    const val NOTIFICATION_CHANNEL_ID = "sms_forwarder_channel"
    const val NOTIFICATION_CHANNEL_NAME = "短信转发服务"
    const val NOTIFICATION_ID = 1423

    // Preference keys
    const val PREFS_NAME = "app_config"
    const val PREF_ENABLED = "enabled"
    const val PREF_START_ON_BOOT = "start_on_boot"
    const val PREF_CHANNELS = "channels"
    const val PREF_KEYWORD_CONFIGS = "keyword_configs"
    const val PREF_SHOW_RECEIVER_PHONE = "show_receiver_phone"
    const val PREF_SHOW_SENDER_PHONE = "show_sender_phone"
    const val PREF_HIGHLIGHT_VERIFICATION_CODE = "highlight_verification_code"
    const val PREF_CUSTOM_SIM1_PHONE = "custom_sim1_phone"
    const val PREF_CUSTOM_SIM2_PHONE = "custom_sim2_phone"

    // Network
    const val NETWORK_DEBOUNCE_MS = 2000L
    const val CALL_TIMEOUT_SECONDS = 20L
    const val CONNECT_TIMEOUT_SECONDS = 10L
    const val READ_TIMEOUT_SECONDS = 20L
}
```

文件 4：SmsForegroundService.kt（全部 264 行）

```kotlin
/*
 * 短信转发助手
 * 版本：v2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.Build
import android.os.IBinder
import android.provider.Telephony
import android.util.Log
import androidx.core.app.NotificationCompat

class SmsForegroundService : Service() {

    companion object {
        private const val TAG = "SmsForegroundService"
    }

    private val smsReceiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            if (intent.action == Telephony.Sms.Intents.SMS_RECEIVED_ACTION) {
                Log.d(TAG, "SMS received")
            }
        }
    }

    private val networkReceiver = NetworkChangeReceiver()

    override fun onCreate() {
        super.onCreate()
        Log.d(TAG, "Service created")

        createNotificationChannel()
        startForeground(Constants.NOTIFICATION_ID, createNotification())
```

```
        registerSmsReceiver()
        registerNetworkReceiver()
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        Log.d(TAG, "Service started")
        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder? {
        return null
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "Service destroyed")

        try {
            unregisterReceiver(smsReceiver)
        } catch (e: Exception) {
            Log.e(TAG, "Error unregistering SMS receiver", e)
        }

        try {
            unregisterReceiver(networkReceiver)
        } catch (e: Exception) {
            Log.e(TAG, "Error unregistering network receiver", e)
        }
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                Constants.NOTIFICATION_CHANNEL_ID,
                Constants.NOTIFICATION_CHANNEL_NAME,
                NotificationManager.IMPORTANCE_LOW
            ).apply {
                description = "短信转发服务运行通知"
                setShowBadge(false)
                setSound(null, null)
                enableVibration(false)
            }

            val manager = getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager
            manager.createNotificationChannel(channel)
        }
    }

    private fun createNotification(): Notification {
        return NotificationCompat.Builder(this, Constants.NOTIFICATION_CHANNEL_ID)
            .setContentTitle("短信转发助手")
            .setContentText("服务运行中")
```

```
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setOngoing(true)
            .setPriority(NotificationCompat.PRIORITY_LOW)
            .build()
    }

    private fun registerSmsReceiver() {
        val filter = IntentFilter(Telephony.Sms.Intents.SMS_RECEIVED_ACTION)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            registerReceiver(smsReceiver, filter, RECEIVER_EXPORTED)
        } else {
            registerReceiver(smsReceiver, filter)
        }
        Log.d(TAG, "SMS receiver registered")
    }

    private fun registerNetworkReceiver() {
        val filter = IntentFilter()
        filter.addAction("android.net.conn.CONNECTIVITY_CHANGE")
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            registerReceiver(networkReceiver, filter, RECEIVER_NOT_EXPORTED)
        } else {
            registerReceiver(networkReceiver, filter)
        }
        Log.d(TAG, "Network receiver registered")
    }
}
```

文件 5：models.kt（全部 29 行）

```
/*
 * 短信转发助手
 * 版本：V2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
 */

package com.lanbing.smsforwarder

/**
 * Shared model types for channels and keyword rules.
 */
enum class ChannelType { WECHAT, DINGTALK, FEISHU, GENERIC_WEBHOOK }

data class Channel(
    val id: String,
    val name: String,
```

```
    val type: ChannelType,
    val target: String           // webhook URL
)

data class KeywordConfig(
    val id: String,
    val keyword: String, // empty string means match-all
    val channelId: String
)
```

第二部分：后 30 页

文件：MainActivity.kt（后 1500 行）

```
@Composable
fun ChannelCard(
    channel: Channel,
    onEdit: () -> Unit,
    onDelete: () -> Unit
) {
    val typeName = when (channel.type) {
        ChannelType.WECHAT -> "企业微信"
        ChannelType.DINGTALK -> "钉钉"
        ChannelType.FEISHU -> "飞书"
        ChannelType.GENERIC_WEBHOOK -> "通用 Webhook"
    }

    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column(modifier = Modifier.weight(1f)) {
                    Text(
                        text = channel.name,
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        text = typeName,
```

```
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        text = channel.target,
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        maxLines = 1
                    )
                }
                Row {
                    IconButton(onClick = onEdit) {
                        Icon(Icons.Default.Edit, contentDescription = "编辑")
                    }
                    IconButton(onClick = onDelete) {
                        Icon(Icons.Default.Delete, contentDescription = "删除", tint =
MaterialTheme.colorScheme.error)
                    }
                }
            }
        }
    }
}

@Composable
fun ChannelEditDialog(
    channel: Channel?,
    onDismiss: () -> Unit,
    onSave: (Channel) -> Unit
) {
    var name by remember { mutableStateOf(channel?.name ?: "") }
    var type by remember { mutableStateOf(channel?.type ?: ChannelType.WECHAT) }
    var target by remember { mutableStateOf(channel?.target ?: "") }
    var showTypeDropdown by remember { mutableStateOf(false) }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(if (channel == null) "添加通道" else "编辑通道") },
        text = {
            Column(
                verticalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                OutlinedTextField(
                    value = name,
                    onValueChange = { name = it },
                    label = { Text("通道名称") },
                    modifier = Modifier.fillMaxWidth(),
                    singleLine = true
                )
                ExposedDropdownMenuBox(
                    expanded = showTypeDropdown,
                    onExpandedChange = { showTypeDropdown = !showTypeDropdown }
```

```
                ) {
                    OutlinedTextField(
                        value = when (type) {
                            ChannelType.WECHAT -> "企业微信"
                            ChannelType.DINGTALK -> "钉钉"
                            ChannelType.FEISHU -> "飞书"
                            ChannelType.GENERIC_WEBHOOK -> "通用 Webhook"
                        },
                        onValueChange = {},
                        readOnly = true,
                        label = { Text("通道类型") },
                        trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded =
showTypeDropdown) },
                        modifier = Modifier
                            .fillMaxWidth()
                            .menuAnchor()
                    )
                    ExposedDropdownMenu(
                        expanded = showTypeDropdown,
                        onDismissRequest = { showTypeDropdown = false }
                    ) {
                        ChannelType.values().forEach { channelType ->
                            DropdownMenuItem(
                                text = {
                                    Text(
                                        when (channelType) {
                                            ChannelType.WECHAT -> "企业微信"
                                            ChannelType.DINGTALK -> "钉钉"
                                            ChannelType.FEISHU -> "飞书"
                                            ChannelType.GENERIC_WEBHOOK -> "通用 Webhook"
                                        }
                                    )
                                },
                                onClick = {
                                    type = channelType
                                    showTypeDropdown = false
                                }
                            )
                        }
                    }
                }
                OutlinedTextField(
                    value = target,
                    onValueChange = { target = it },
                    label = { Text("Webhook URL") },
                    modifier = Modifier.fillMaxWidth(),
                    singleLine = true,
                    keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Uri)
                )
                Text(
                    "请确保 URL 以 http:// 或 https:// 开头",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
```

```
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    if (name.isNotEmpty() && target.isNotEmpty()) {
                        onSave(
                            Channel(
                                id = channel?.id ?: java.util.UUID.randomUUID().toString(),
                                name = name,
                                type = type,
                                target = target
                            )
                        )
                    }
                },
                enabled = name.isNotEmpty() && target.isNotEmpty()
            ) {
                Text("保存")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
                Text("取消")
            }
        }
    )
}

@Composable
fun SettingsTab(prefs: android.content.SharedPreferences, context: Context) {
    var customSim1Phone by remember {
        mutableStateOf(prefs.getString(Constants.PREF_CUSTOM_SIM1_PHONE, "") ?: "")
    }
    var customSim2Phone by remember {
        mutableStateOf(prefs.getString(Constants.PREF_CUSTOM_SIM2_PHONE, "") ?: "")
    }
    var showReceiverPhone by remember {
        mutableStateOf(prefs.getBoolean(Constants.PREF_SHOW_RECEIVER_PHONE, true))
    }
    var showSenderPhone by remember {
        mutableStateOf(prefs.getBoolean(Constants.PREF_SHOW_SENDER_PHONE, true))
    }
    var highlightCode by remember {
        mutableStateOf(prefs.getBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE, true))
    }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
```

```
    ) {
        item {
            Card(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(Icons.Default.Phone, contentDescription = null)
                        Spacer(modifier = Modifier.width(12.dp))
                        Text(
                            "SIM 卡号码配置",
                            style = MaterialTheme.typography.titleMedium,
                            fontWeight = FontWeight.Bold
                        )
                    }
                    Spacer(modifier = Modifier.height(16.dp))
                    Text(
                        "如果无法自动获取 SIM 卡号码，请手动输入",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    OutlinedTextField(
                        value = customSim1Phone,
                        onValueChange = {
                            customSim1Phone = it
                            prefs.edit().putString(Constants.PREF_CUSTOM_SIM1_PHONE,
it).apply()
                        },
                        label = { Text("SIM1 手机号码") },
                        modifier = Modifier.fillMaxWidth(),
                        singleLine = true,
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone)
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    OutlinedTextField(
                        value = customSim2Phone,
                        onValueChange = {
                            customSim2Phone = it
                            prefs.edit().putString(Constants.PREF_CUSTOM_SIM2_PHONE,
it).apply()
                        },
                        label = { Text("SIM2 手机号码") },
                        modifier = Modifier.fillMaxWidth(),
                        singleLine = true,
                        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone)
```

```
                )
            }
        }
    }

    item {
        Card(
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp)
            ) {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Icon(Icons.Default.Message, contentDescription = null)
                    Spacer(modifier = Modifier.width(12.dp))
                    Text(
                        "消息格式",
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.Bold
                    )
                }
                Spacer(modifier = Modifier.height(16.dp))
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("显示本机号码", style = MaterialTheme.typography.bodyMedium)
                    Switch(
                        checked = showReceiverPhone,
                        onCheckedChange = {
                            showReceiverPhone = it
                            prefs.edit().putBoolean(Constants.PREF_SHOW_RECEIVER_PHONE,
it).apply()
                        }
                    )
                }
                Spacer(modifier = Modifier.height(8.dp))
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text("显示发送者号码", style = MaterialTheme.typography.bodyMedium)
                    Switch(
                        checked = showSenderPhone,
                        onCheckedChange = {
                            showSenderPhone = it
```

```
                                prefs.edit().putBoolean(Constants.PREF_SHOW_SENDER_PHONE,
it).apply()
                            }
                        )
                    }
                    Spacer(modifier = Modifier.height(8.dp))
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceBetween,
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Text("突出显示验证码", style = MaterialTheme.typography.bodyMedium)
                        Switch(
                            checked = highlightCode,
                            onCheckedChange = {
                                highlightCode = it

 prefs.edit().putBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE, it).apply()
                            }
                        )
                    }
                }
            }
        }

        item {
            Card(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp)
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Icon(Icons.Default.Info, contentDescription = null)
                        Spacer(modifier = Modifier.width(12.dp))
                        Text(
                            "关于",
                            style = MaterialTheme.typography.titleMedium,
                            fontWeight = FontWeight.Bold
                        )
                    }
                    Spacer(modifier = Modifier.height(12.dp))
                    OutlinedButton(
                        onClick = { /* TODO: 测试规则 */ },
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Icon(Icons.Default.PlayArrow, contentDescription = null)
                        Spacer(modifier = Modifier.width(8.dp))
```

```
                        Text("测试规则")
                    }
                }
            }
        }
    }
}

@Composable
fun LogTab(context: Context, refreshKey: Int) {
    var logs by remember(refreshKey) { mutableStateOf(LogStore.loadLogs(context)) }
    var showClearDialog by remember { mutableStateOf(false) }

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp),
        reverseLayout = true
    ) {
        if (logs.isEmpty()) {
            item {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.secondaryContainer
                    )
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(24.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Icon(
                            Icons.Default.List,
                            contentDescription = null,
                            modifier = Modifier.size(48.dp),
                            tint = MaterialTheme.colorScheme.onSecondaryContainer
                        )
                        Spacer(modifier = Modifier.height(12.dp))
                        Text(
                            "暂无日志",
                            style = MaterialTheme.typography.titleLarge,
                            fontWeight = FontWeight.Bold
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            "收到短信后会在这里显示转发记录",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSecondaryContainer
                        )
                    }
```

```
                }
            }
        }

        items(logs) { log ->
            LogEntryCard(log)
        }

        item {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(12.dp)
            ) {
                OutlinedButton(
                    onClick = { logs = LogStore.loadLogs(context) },
                    modifier = Modifier.weight(1f)
                ) {
                    Icon(Icons.Default.Refresh, contentDescription = null)
                    Spacer(modifier = Modifier.width(8.dp))
                    Text("刷新")
                }
                Button(
                    onClick = { showClearDialog = true },
                    modifier = Modifier.weight(1f),
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.errorContainer,
                        contentColor = MaterialTheme.colorScheme.onErrorContainer
                    )
                ) {
                    Icon(Icons.Default.Delete, contentDescription = null)
                    Spacer(modifier = Modifier.width(8.dp))
                    Text("清空")
                }
            }
        }

        item {
            Spacer(modifier = Modifier.height(80.dp))
        }
    }

    if (showClearDialog) {
        AlertDialog(
            onDismissRequest = { showClearDialog = false },
            title = { Text("确认清空") },
            text = { Text("确定要清空所有日志吗？此操作不可恢复。") },
            confirmButton = {
                Button(
                    onClick = {
                        LogStore.clearLogs(context)
                        logs = emptyList()
                        showClearDialog = false
                    },
```

```
                    colors = ButtonDefaults.buttonColors(
                        containerColor = MaterialTheme.colorScheme.errorContainer,
                        contentColor = MaterialTheme.colorScheme.onErrorContainer
                    )
                ) {
                    Text("清空")
                }
            },
            dismissButton = {
                TextButton(onClick = { showClearDialog = false }) {
                    Text("取消")
                }
            }
        )
    }
}

@Composable
fun LogEntryCard(log: String) {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(12.dp)
        ) {
            Text(
                text = log,
                style = MaterialTheme.typography.bodySmall,
                fontFamily = androidx.compose.ui.text.font.FontFamily.Monospace
            )
        }
    }
}

@Composable
fun AboutDialog(onDismiss: () -> Unit) {
    val context = LocalContext.current

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("关于短信转发助手") },
        text = {
            Column(
                verticalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.primaryContainer
                    )
                ) {
```

```
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(20.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Icon(
                        Icons.Default.Sms,
                        contentDescription = null,
                        modifier = Modifier.size(64.dp),
                        tint = MaterialTheme.colorScheme.primary
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    Text(
                        "短信转发助手",
                        style = MaterialTheme.typography.headlineSmall,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(4.dp))
                    Text(
                        "版本 2.7.2",
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onPrimaryContainer
                    )
                }
            }

            Text(
                "一款功能完善的 Android 短信转发工具，支持企业微信、钉钉、飞书和通用 Webhook 等多种转发渠道，支持关键词过滤、验证码自动提取、双卡识别等功能。",
                style = MaterialTheme.typography.bodyMedium
            )

            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                Text(
                    "功能特点：",
                    style = MaterialTheme.typography.titleSmall,
                    fontWeight = FontWeight.Bold
                )
                Text(
                    "• 支持多种转发渠道（企业微信、钉钉、飞书、通用 Webhook）\n" +
                    "• 关键词过滤，空关键词转发全部\n" +
                    "• 双卡 SIM 卡识别，自定义本机号码\n" +
                    "• 验证码自动提取与突出显示\n" +
                    "• 完整的日志记录与查询\n" +
                    "• Material Design 3 界面，支持深色模式",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }

            HorizontalDivider()
```

```
                Text(
                    "© 2026 华昊科技有限公司",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    textAlign = androidx.compose.ui.text.style.TextAlign.Center,
                    modifier = Modifier.fillMaxWidth()
                )
            }
        },
        confirmButton = {
            Button(
                onClick = onDismiss,
                modifier = Modifier.fillMaxWidth(),
                shape = androidx.compose.foundation.shape.RoundedCornerShape(12.dp),
                contentPadding = androidx.compose.foundation.layout.PaddingValues(horizontal
= 24.dp, vertical = 12.dp)
            ) {
                Text("关闭")
            }
        }
    )
}
```

说明

本文件包含短信转发助手（V2.7.2）的源程序鉴别材料，包含：

1. **前 30 页**：
    - MainActivity.kt（前 600 行）
    - SmsReceiver.kt（全部 586 行）
    - Constants.kt（全部 58 行）
    - SmsForegroundService.kt（全部 264 行）
    - models.kt（全部 29 行）
2. **后 30 页**：
    - MainActivity.kt（后 1500 行）

所有文件均包含完整的版权信息。

著作权人：华昊科技有限公司
开发者：王士辉
版本：V2.7.2
联系邮箱：huahao@email.cn